

C++

C++/C

Чем отличается C/C++ от других языков программирования

Какое место занимает C/C++ в IT отрасли

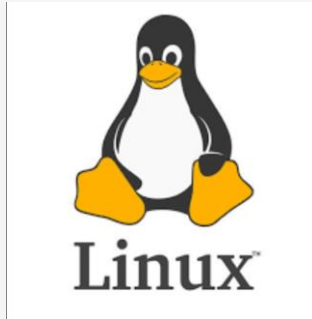
Особенности сборки C/C++ кода

2019 and 1979



Power and connectivity

C++/C



C++/C

```
#include <iostream>
```

```
int main()  
{  
    std::cout << "Hello world!\n";  
}
```

???



ConsoleApplication1.exe
ConsoleApplication1.pdb

???



Microsoft Visual Studio Debug Console
Hello World!

Ассемблер

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	is program canno
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	t be run in DOS
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	mode....\$.....
00000030	00	00	00	00	00	00	00	00	00	00	00	00	F0	00	00	00	..°.°.í!..Lí!Th
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	is program canno
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	t be run in DOS
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	mode....\$.....
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	..t.\$ô.ywô.ywô.yw
00000080	90	74	17	24	D4	15	79	77	D4	15	79	77	D4	15	79	77	YmêwB.ywA vE.yw
00000090	DD	6D	EA	77	DE	15	79	77	C1	6A	7C	76	C6	15	79	77	Aj}vB.ywA vE.yw
000000A0	C1	6A	7D	76	DE	15	79	77	C1	6A	7A	76	D7	15	79	77	Aj}vB.ywA vE.yw
000000B0	C1	6A	78	76	D3	15	79	77	9F	6D	78	76	D6	15	79	77	Aj}vB.ywA vE.yw
000000C0	D4	15	78	77	D4	15	79	77	E2	95	70	76	D5	15	79	77	ô.xw".ywA vE.yw
000000D0	E2	95	86	77	D5	15	79	77	E2	95	7B	76	D5	15	79	77	A*twô.ywA vE.yw
000000E0	52	69	63	68	D4	15	79	77	00	00	00	00	00	00	00	00	Richô.yw.....
000000F0	50	45	00	00	64	86	06	00	2D	11	27	65	00	00	00	00	PE..dt..-.'e....
00000100	00	00	00	00	F0	00	22	00	0B	02	0E	25	00	10	00	00	ô.".....
00000110	00	22	00	00	00	00	00	00	D0	14	00	00	00	10	00	00	.."......ô.....
00000120	00	00	00	40	01	00	00	00	00	10	00	00	00	02	00	00	...@.....
00000130	06	00	00	00	00	00	00	00	06	00	00	00	00	00	00	00	
00000140	00	80	00	00	00	04	00	00	00	00	00	00	03	00	60	81	..E.....
00000150	00	00	10	00	00	00	00	00	00	10	00	00	00	00	00	00	
00000160	00	00	10	00	00	00	00	00	00	10	00	00	00	00	00	00	
00000170	00	00	00	00	10	00	00	00	00	00	00	00	00	00	00	00	
00000180	6C	29	00	00	C8	00	00	00	00	60	00	00	E0	01	00	00	l)..E.....'â...
00000190	00	50	00	00	98	01	00	00	00	00	00	00	00	00	00	00	..P.."......
000001A0	00	70	00	00	30	00	00	00	F0	23	00	00	70	00	00	00	..p..0...ô#..p...
000001B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001C0	00	00	00	00	00	00	00	00	B0	22	00	00	40	01	00	00	".@.....
000001D0	00	00	00	00	00	00	00	00	00	20	00	00	F0	01	00	00	ô.....
000001E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001F0	00	00	00	00	00	00	00	00	2E	74	65	78	74	00	00	00	text...
00000200	A9	0F	00	00	00	10	00	00	00	10	00	00	00	04	00	00	@.....
00000210	00	00	00	00	00	00	00	00	00	00	00	00	20	00	00	60	`.....
00000220	2E	72	64	61	74	61	00	00	70	12	00	00	00	20	00	00	..rdata..p....
00000230	00	14	00	00	00	14	00	00	00	00	00	00	00	00	00	00	
00000240	00	00	00	00	40	00	00	40	2E	64	61	74	61	00	00	00	@..@.data...
00000250	38	06	00	00	00	40	00	00	02	00	00	00	28	00	00	00	@.....@.....(..
00000260	00	00	00	00	00	00	00	00	00	00	00	00	40	00	00	C0	@..A.....
00000270	2E	70	64	61	74	61	00	00	98	01	00	00	00	50	00	00	..pdata..".....P..
00000280	00	02	00	00	00	2A	00	00	00	00	00	00	00	00	00	00	*.....
00000290	00	00	00	00	40	00	00	40	2E	72	73	72	63	00	00	00	@..@.rsrc...

Язык ассемблера (англ. assembly language) — представление команд процессора в виде, доступном для чтения человеком. С. Wikipedia

В языке ассемблера все вычисления производятся в регистрах.

Отдельный команды для записи из регистров в память и наоборот (Load store)

Управление control flow через различные conditional jump

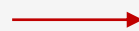
Каждая команда ассемблера кодируется в несколько байт

```
1
2 float square(float num) {
3     if (num > 0.0)
4     {
5         return 0.0;
6     }
7     return num * num;
8 }
```



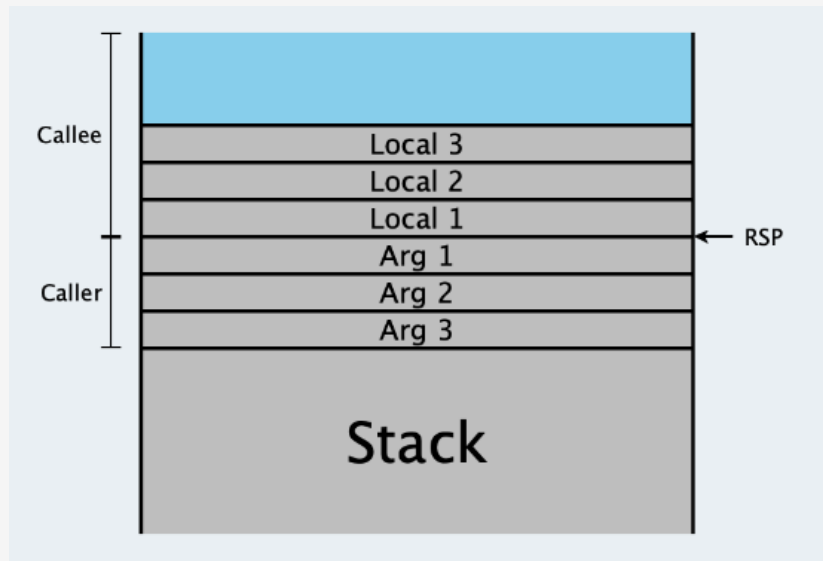
```
1  ∨ square(float):
2      addi    sp,sp,-32
3      sw      ra,28(sp)
4      sw      s0,24(sp)
5      addi    s0,sp,32
6      fsw     fa0,-20(s0)
7      flw     fa5,-20(s0)
8      fmv.s.x fa4,zero
9      fgt.s   a5,fa5,fa4
10     beq     a5,zero,.L6
11     fmv.s.x fa5,zero
12     j       .L4
13  ∨ .L6:
14     flw     fa5,-20(s0)
15     fmul.s  fa5,fa5,fa5
16  ∨ .L4:
17     fmv.s   fa0,fa5
18     lw      ra,28(sp)
19     lw      s0,24(sp)
20     addi    sp,sp,32
21     jr      ra
```

add r0 r1 r2



0	1	1	0	0	1	0	0	1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

А где же стек (stack) и куча (heap)?



Stack используется очень часто поэтому современные ISA имеют специальные ассемблерные команды для работы со стеком

Stack это отдельная область памяти

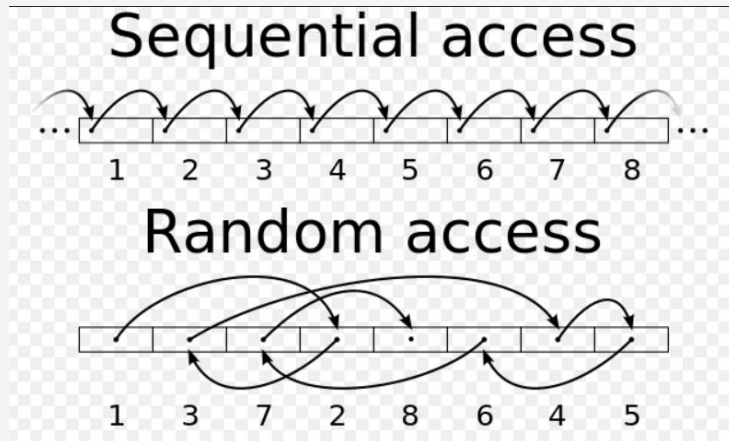
Calling convection

Stack corruption

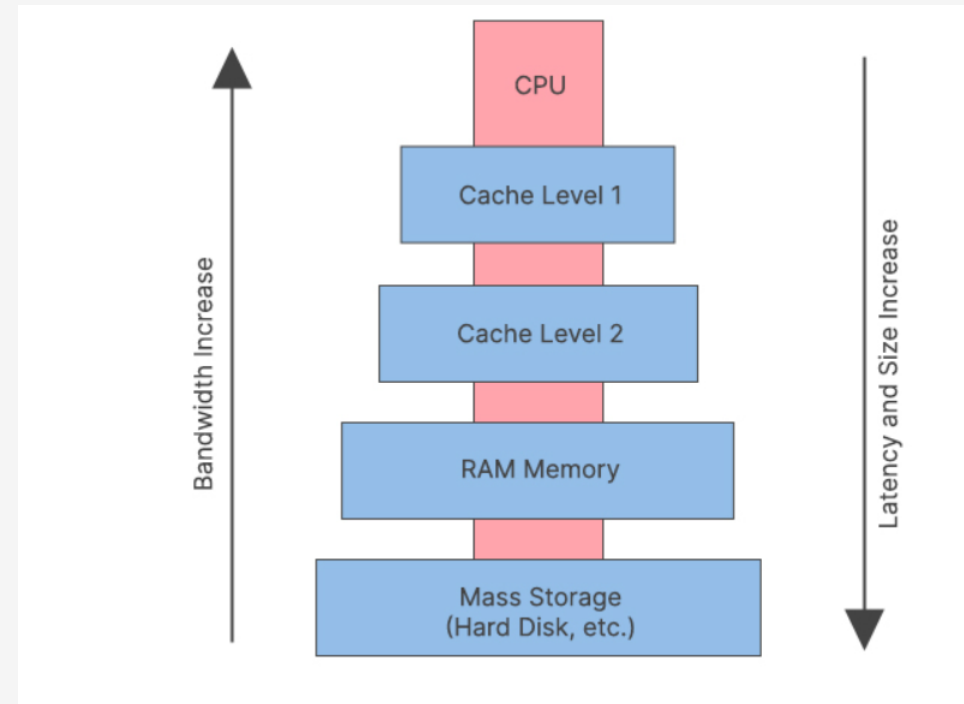
Cache miss

Чтение из памяти длится порядка сотни тактов процессора

Cache процессора строится на предположении что доступ к памяти программы будет локальным

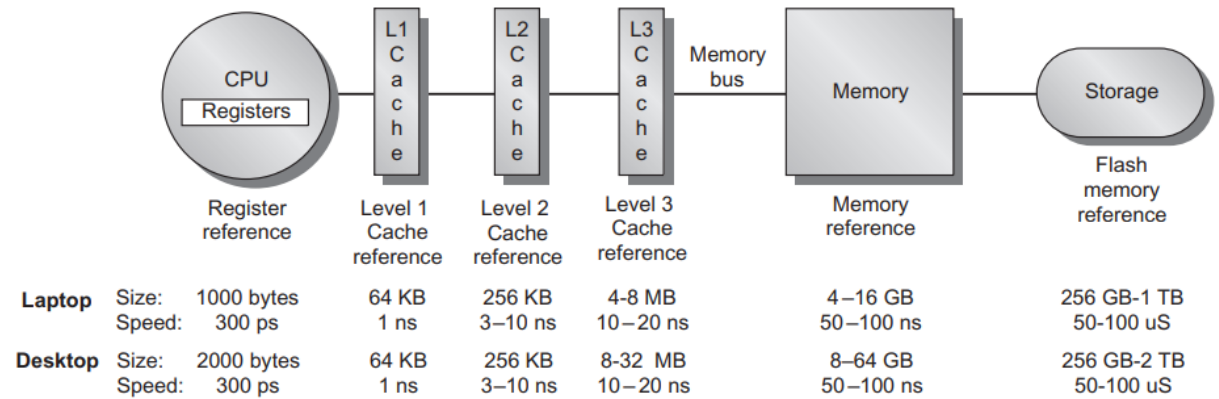
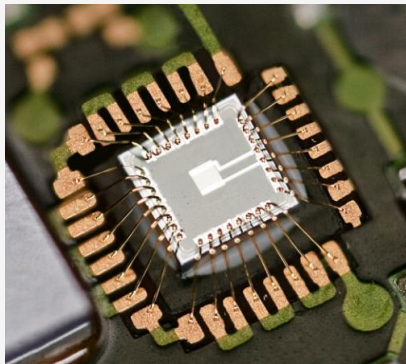
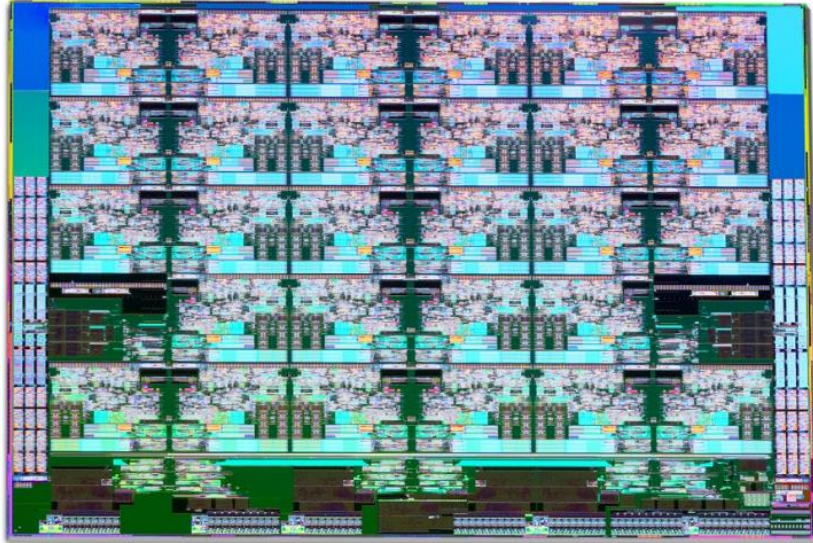


Как работают кеши в многоядерном процессоре? (Cache coherence)



[Sorting Algorithms: Speed Is Found In The Minds of People - Andrei Alexandrescu - CppCon 2019](#)

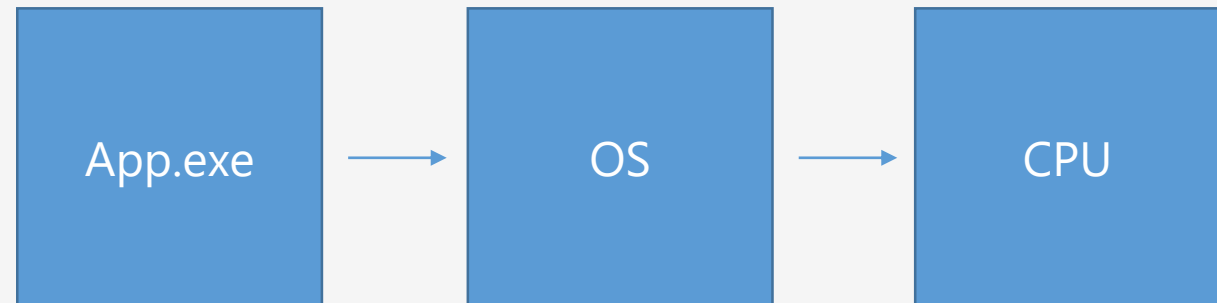
CPU



(B)

Memory hierarchy for a laptop or a desktop

[Computer Architecture: A Quantitative Approach Book](#)
[Computer Architecture - Lecture 1: Introduction and Basics](#)
[\(ETH Zürich, Fall 2020\)](#)



CPP

C++/C может напрямую компилироваться в бинарный код и запускаться напрямую на embedded девайсах

C++ напрямую компилируется в бинарный исполняемые файлы OS



C++/C может напрямую компилироваться в бинарный код и запускаться напрямую на embedded девайсах

C++ напрямую компилируется в бинарные исполняемые файлы OS

```
hello_world.py
1  if __name__ == "__main__":
2      print("hello world!")
```

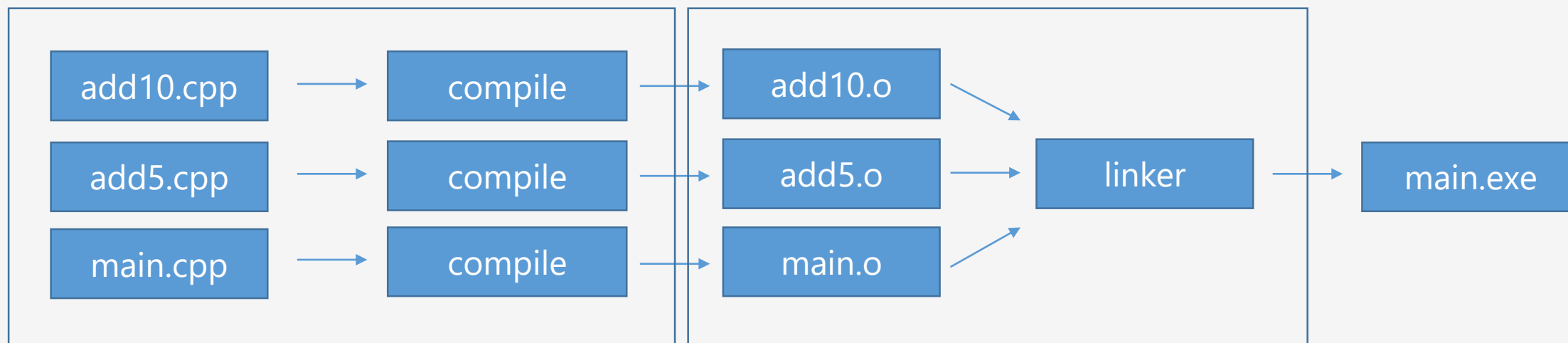
```
PS H:\test> & h:/ssd/neyronki/pix2pix/pix2pixBoost/python/python-3.10.7-embed-amd64/python.exe h:/test/hello_world.py
hello world!
```

Python.exe написан на pure C

Многие языки абстрагируют программиста от прямой работы с памятью

C/C++ не единственные языки которые поддерживают прямую компиляцию в бинарные файлы

Процесс компиляции C++/C кода



Компиляция может выполняться параллельно

Compile-time связывание или static linking

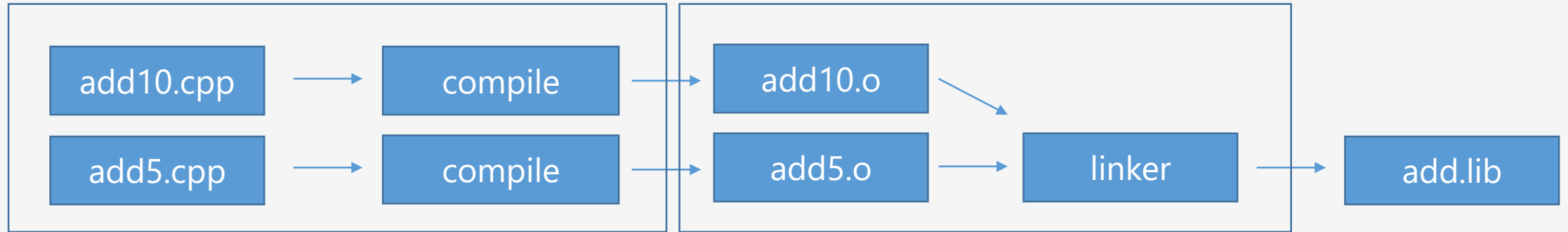
Как компилятор узнает о add5 и add10 функциях если компиляция происходит параллельно?

```
float add10(float x)
{
    return x + 10.0f;
}
```

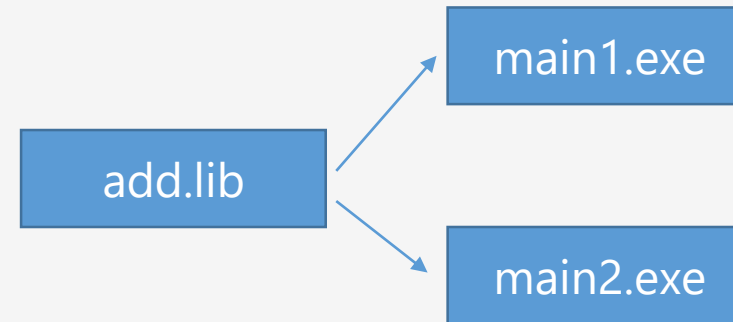
```
float add5(float x)
{
    return x + 5.0f;
}
```

```
int main()
{
    float t = 0;
    std::cin >> t;
    std::cout << add10(add5(t));
    return 0;
}
```

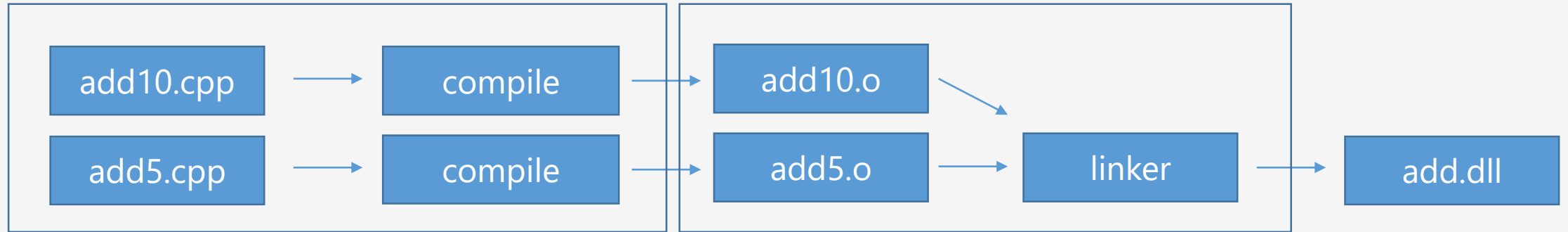
Static library linking



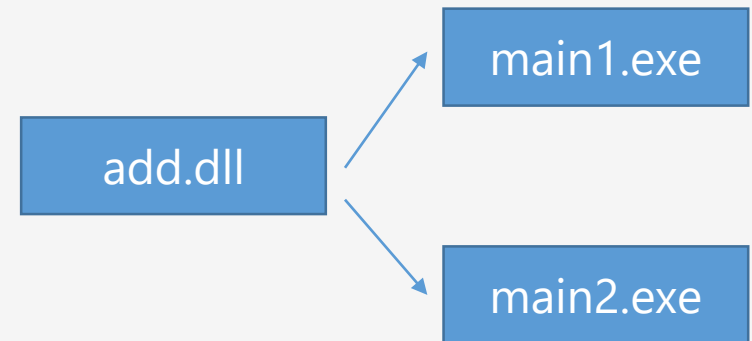
```
int main()
{
    float t = 0;
    std::cin >> t;
    std::cout << add10(add5(t));
    return 0;
}
```



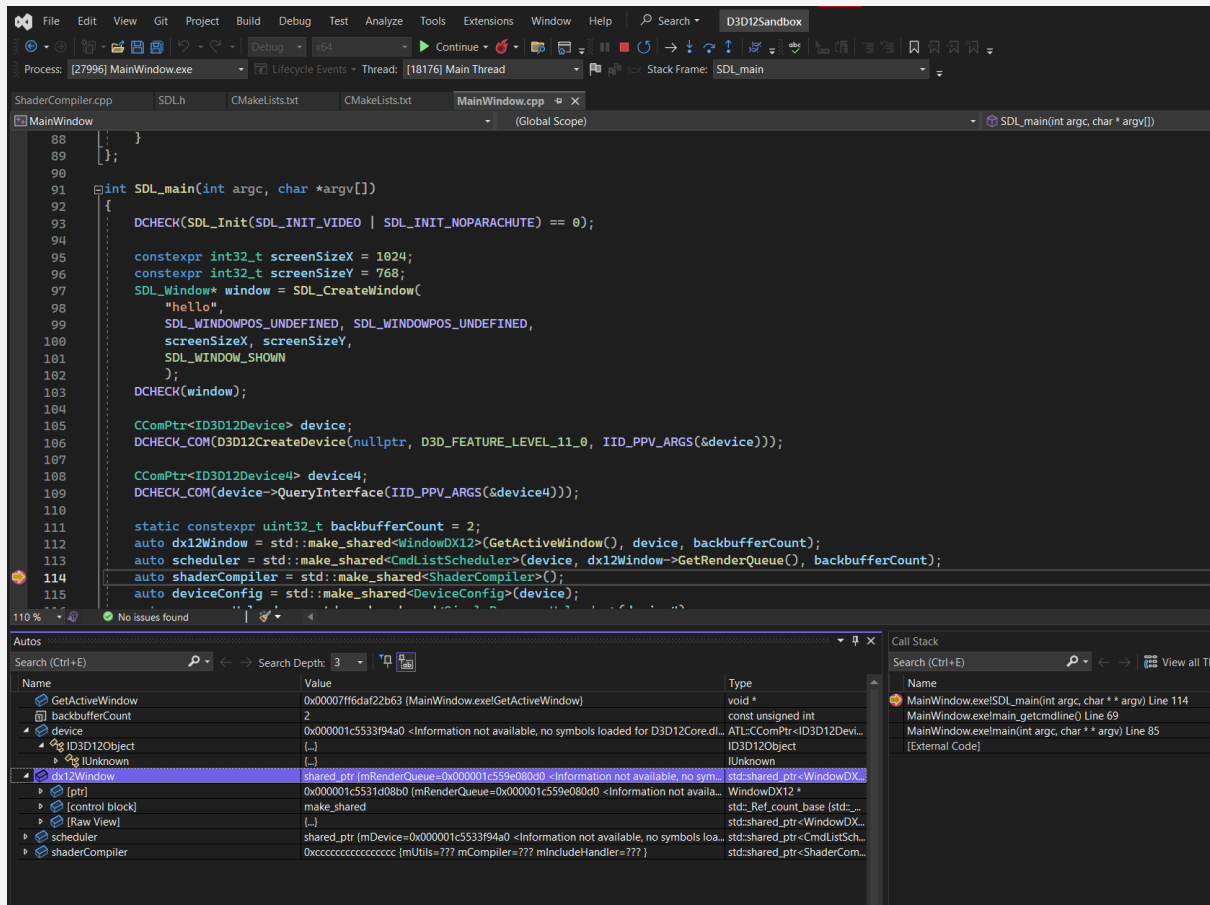
Dynamic library linking



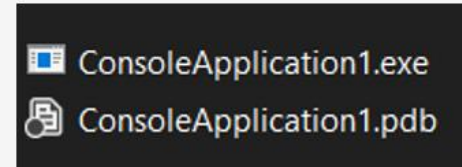
```
int main()
{
    HMODULE dynamic_lib = LoadLibraryW(L"add.dll");
    void* ptr = GetProcAddress(dynamic_lib, "add10");
    //.....
}
```



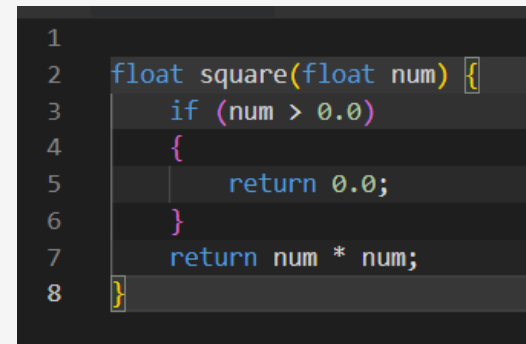
Debug Symbols



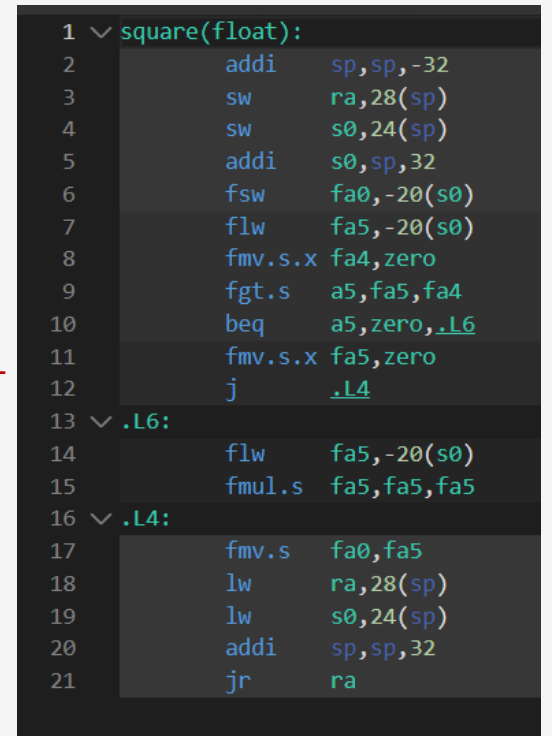
The screenshot shows the Visual Studio IDE with the file `MainWindow.cpp` open. The code is in C++ and includes SDL headers. It defines a `SDL_main` function that initializes SDL, creates a window, and sets up a DirectX 12 environment. The `Autos` window at the bottom shows the current state of variables, including `dx12Window` which is a `shared_ptr<mRenderQueue>`.



.pdb – файлы с Debug Symbols



```
1
2 float square(float num) {
3     if (num > 0.0)
4     {
5         return 0.0;
6     }
7     return num * num;
8 }
```



```
1  square(float):
2      addi    sp,sp,-32
3      sw      ra,28(sp)
4      sw      s0,24(sp)
5      addi    s0,sp,32
6      fsw     fa0,-20(s0)
7      flw     fa5,-20(s0)
8      fmv.s.x fa4,zero
9      fgt.s   a5,fa5,fa4
10     beq     a5,zero,.L6
11     fmv.s.x fa5,zero
12     j       .L4
13  .L6:
14     flw     fa5,-20(s0)
15     fmul.s  fa5,fa5,fa5
16  .L4:
17     fmv.s   fa0,fa5
18     lw      ra,28(sp)
19     lw      s0,24(sp)
20     addi    sp,sp,32
21     jr      ra
```


Отличие Debug, Release, RelWithDebInfo, сборок

Без оптимизаций

```
float add2(float num) {  
    return num + 2;  
}  
  
float add2_t(float num) {  
    num = num + 2;  
    return num;  
}
```

```
1  __real@40000000 DD 04000000r          ; 2  
2  
3  num$ = 8  
4  float add2(float) PROC                ; add2  
5      movss  DWORD PTR [rsp+8], xmm0  
6      movss  xmm0, DWORD PTR num$[rsp]  
7      addss  xmm0, DWORD PTR __real@40000000  
8      ret    0  
9  float add2(float) ENDP                ; add2  
10  
11 num$ = 8  
12 float add2_t(float) PROC              ; add2_t  
13     movss  DWORD PTR [rsp+8], xmm0  
14     movss  xmm0, DWORD PTR num$[rsp]  
15     addss  xmm0, DWORD PTR __real@40000000  
16     movss  DWORD PTR num$[rsp], xmm0  
17     movss  xmm0, DWORD PTR num$[rsp]  
18     ret    0  
19 float add2_t(float) ENDP              ; add2_t
```

С оптимизацией

```
1  __real@40000000 DD 04000000r          ; 2  
2  
3  num$ = 8  
4  float add2(float) PROC                ; add2, COMDAT  
5      addss  xmm0, DWORD PTR __real@40000000  
6      ret    0  
7  float add2(float) ENDP                ; add2  
8  
9  num$ = 8  
10 float add2_t(float) PROC              ; add2_t, COMDAT  
11     addss  xmm0, DWORD PTR __real@40000000  
12     ret    0  
13 float add2_t(float) ENDP              ; add2_t
```

<https://godbolt.org/>

- Позволяет увидеть ассемблерный код различных компиляторов в читаемой форме

C++/C

The purpose of a programming language is to help express ideas in code.

In that, a programming language performs two related tasks:

- it provides a vehicle for the programmer to specify actions to be executed by the machine,

 - "close to the machine"

 - direct mappings of built-in operations and types to hardware to provide efficient memory use and efficient low-level operations

- it provides a set of concepts for the programmer to use when thinking about what can be done.

 - "close to the problem to be solved"

 - affordable and flexible abstraction mechanisms

C++ is a language for developing and using elegant and efficient abstractions

[The C++ Programming Language, 4th Edition, Bjarne Stroustrup Book](#)

C++/C

The general ideals for design and programming can be expressed simply:

- Express ideas directly in code.

- Express independent ideas independently in code.

- Represent relationships among ideas directly in code.

- Combine ideas expressed in code freely – where and only where combinations make sense.

- Express simple ideas simply

[The C++ Programming Language, 4th Edition, Bjarne Stroustrup Book](#)

C++/C

The C++ language features most directly support four programming styles:

Procedural programming

This is programming focused on processing and the design of suitable data structures.

It is what C was designed to support

Data abstraction

This is programming focused on the design of interfaces, hiding implementation details in general and representations in particular.

Object-oriented programming

This is programming focused on the design, implementation, and use of class hierarchies.

Generic programming

This is programming focused on the design, implementation, and use of general algorithms.

Here, "general" means that an algorithm can be designed to accept a wide variety of types

[The C++ Programming Language, 4th Edition, Bjarne Stroustrup Book](#)